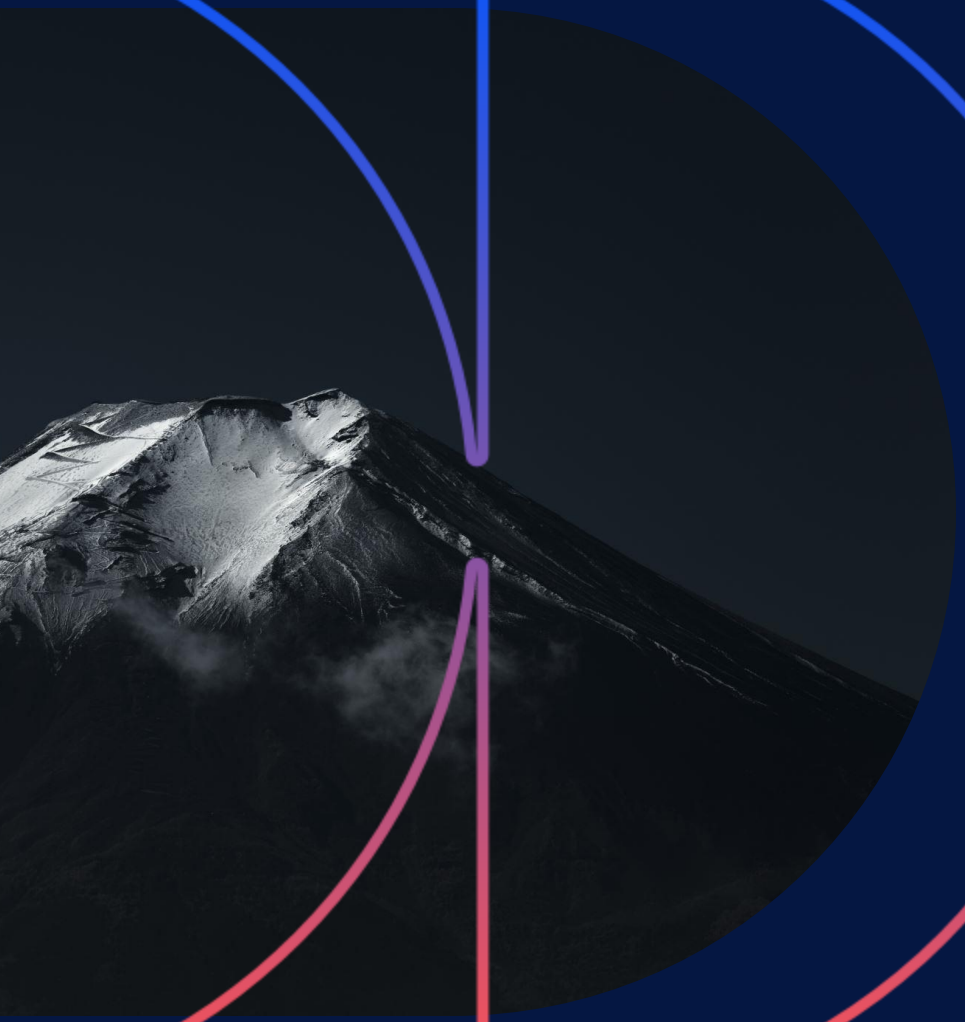


néosoft

# Django Ninja

# Votre API nette et sans bavure ?

27/04/2026 @ Néosoft Rennes

Restreint Néosoft

# Remerciements

- Néosoft
- Luc Sorel-Giffo
- Alexandre Cuffel
- Michel Caradec

# Introduction au RETEX

Vous aimez **Django** mais vous avez préféré un autre framework (ou langage ?!) pour écrire votre API ?

Ce retour d'expérience sur **Django Ninja** vous fera peut-être changer d'avis.

Après un tour d'horizon des frameworks web API Python permettant de planter le décor, découvrez les caractéristiques du framework, les difficultés rencontrées en 10 semaines d'utilisation et les contournements possibles pour devenir un véritable ninja des APIs.

Restreint Néosoft

# Déroulement

1. Tour d'horizon des frameworks web Python
2. Les grandes fonctionnalités de Django
3. Les raisons d'utiliser Django Ninja
4. Mise en pratique
5. Conclusions

Restreint Néosoft

néosoft

# Tour d'horizon des frameworks web Python

1

Restreint Néosoft

# Les frameworks agnostiques

néosoft

**Frameworks**

1. Bottle
2. CherryPy
3. Falcon
4. FastAPI
5. Flask
6. Litestar
7. Masonite
8. Pylons
9. Pyramid
10. Quart
11. Sanic
12. Tornado
13. Turbogears
14. Zope

**ORM**

1. Django ORM
2. Gino
3. Ormar
4. Peewee
5. Piccolo
6. Pony ORM
7. SQLAlchemy
8. SQLModel (Pydantic + SQLAlchemy)
9. SQLObject
10. Tortoise ORM

ℹ️ L'ORM Django peut être utilisé de manière autonome (standalone)
Palolo Melchiore : Django ORM Standalone: Querying an existing database

Restreint Néosoft

# Les frameworks basés sur Django

néosoft

**Frameworks**

1. Django Bolt
2. Django Ninja 🥷 📈
3. Django Shinobi (fork) 📉
4. DRF (Django Rest Framework)

**ORM**

1. Django ORM
2. SQLALchemy (django-sorcery 💀 )

Et tous les précédents ?

Restreint Néosoft

# Popularité des frameworks

néosoft

| Position | Nom du framework | Nombre de dl/ mois | Pourcentage |
|---|---|---|---|
| 🥇 | FastAPI | 414 650 450 | 49,8% |
| 🥈 | Flask | 200 587 491 | 24,1% |
| 🥉 | Tornado | 124 462 808 | 15,0% |
| 4 | Django | 44 325 007 | 5,3% |
| 5 | DRF 👀 | 24 256 611 | 2,9% |
| 6 | Bottle | 10 441 841 | 1,3% |
| 7 | Quart | 2 838 439 | 0,3% |
| 8 | Sanic | 2 679 629 | 0,3% |
| 9 | Pyramid | 2 464 097 | 0,3% |
| 10 | Django Ninja 👀 | 2 018 164 | 0,2% |
| 11 | Litestar | 1 484 293 | 0,2% |
| 12 | Falcon | 966 962 | 0,1% |
| 13 | CherryPy | 795 968 | 0,1% |
| 14 | Zope | 138 427 | 0,0% |
| 15 | Masonite | 59 313 | 0,0% |
| 16 | Pylons | 14 457 | 0,0% |
| 17 | Turbogears | 4 378 | 0,0% |
| 18 | Django Bolt 👀 | 2 743 | 0,0% |
| | | 832,191,078 | 100,0% |

Restreint Néosoft

# Popularité des ORM

néosoft

| Position | Nom du framework | Nombre de dl/ mois | Pourcentage |
|---|---|---|---|
| 🥇 | SQLAlchemy | 346 988 659 | 78.9% |
| 🥈 | Django 👀 | 44 325 007 | 10.2% |
| 🥉 | peewee | 32 289 708 | 7.2% |
| 4 | SQLModel (Pydantic + SQLAlchemy) | 14 863 030 | 3.4% |
| 5 | Tortoise ORM | 541 341 | 0.1% |
| 6 | Pony ORM | 248 456 | 0.1% |
| 7 | piccolo | 235 718 | 0.1% |
| 8 | ormar | 215 879 | 0.0% |
| 9 | Gino | 26 722 | 0.0% |
| 10 | SQLObject | 10 534 | 0.0% |
| | | 439 661 365 | 100.0% |

Restreint Néosoft

# Quel framework choisir ?

néosoft

- Cas d'usage : performances, protocoles, ORM et support BDD
- Compétences déjà en place et montée en compétences
- Facilité de MEP

STX Next : *A Beginner's Introduction to Best Python Web Frameworks in 2022*

❓ Which Python web framework has the best performance?

➡️ The one you can finish the project with.

JetBrains : *Which Is the Best Python Web Framework: Django, Flask, or FastAPI?*

Django suits full-stack web apps that need built-in features such as authentication, ORM support, and security.

Huy Nguyen : Dépôt GitHub *Python API Framework Benchmark*

A comprehensive benchmark comparing 5 Python web frameworks with realistic workloads, Docker resource constraints, and automatic resource monitoring.

# Les grandes fonctionnalités de Django

2

# Pourquoi aller vers Django ? 1/2

néosoft

https://www.djangoproject.com/start/overview/

Avec Django, vous pouvez mener à bien le développement d'une application web, de la conception à la mise en ligne, en quelques heures seulement.

Django vous épargne une grande partie des tracas liés au développement web, ce qui vous permet de vous concentrer sur la création de votre application sans avoir à réinventer la roue.

C'est un logiciel gratuit et open source.

Une fondation (Django Software Foundation, DSF) soutient son développement en grande partie grâce à des dons et au travail de 2 763 contributeurs !

Restreint Néosoft

# Pourquoi aller vers Django ? 2/2

néosoft

https://www.djangoproject.com/start/

Fonctionnalités :

1. ORM
2. URL (routes) et vues (*controllers*)
3. Gabarits (Django Template Language)
4. Formulaires
5. Authentification
6. Interface d'administration
7. Internationalisation et localisation (i18n, l10n)
8. Sécurité (CSRF, XSS, SQL injection, etc.)

Restreint Néosoft

# Les raisons d'utiliser Django Ninja

3

# Django Ninja - Caractéristiques

https://django-ninja.dev/

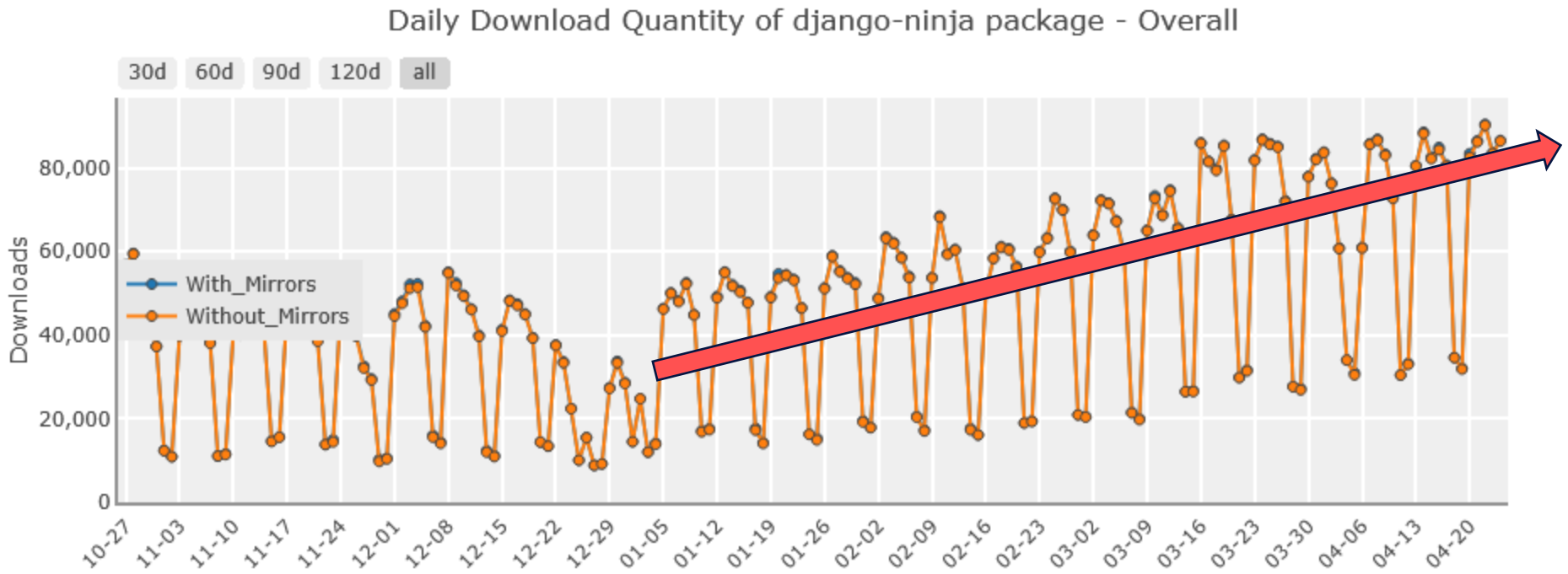
1. Compatible avec Django
2. Facile d'utilisation et intuitif
3. Rapidité d'exécution (Pydantic, async/await)
4. Conforme aux normes : doc Open API (Swagger) ou ReDoc, JSON Schema
5. Prêt pour la production

Mais aussi :

1. Alternative intéressante à Django Rest Framework
2. Popularité croissante

# Django Ninja - Popularité

https://pypistats.org/packages/django-ninja



# Django Ninja – Raisons d'être 1/

```python
# Pour éviter l'accumulation d'injections de dépendances

app = FastAPI()

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

async def get_current_user(token: str = Depends(oauth2_scheme)):
    user = decode(token)
    if not user:
        raise HTTPException(...)
    return user

@app.get("/task/{task_id}", response_model=Task)
def read_user(
        task_id: int,
        db: Session = Depends(get_db),
        current_user: User = Depends(get_current_user),
    ):
        ... use db with current_user ...
```

Restreint Néosoft

# Django Ninja – Raisons d'être 2/

néosoft

Étant donné que le terme ***Model*** est déjà utilisé par l'ORM de Django, cela prête à confusion lorsqu'on l'utilise Pydantic et son ***BaseModel***.

Django Ninja ajoute une **surcouche** à ***pydantic.BaseModel*** ➡️ ***Schema***

Un ***Schema*** sert à la **validation** de données, comme la **sérialisation** et la **désérialisation** des objets Django.

```
# Model
class Employee(models.Model):
    first_name = models.CharField(max_length=100)
    last_name = models.CharField(max_length=100)


# Schemas
class EmployeeIn(Schema):
    first_name: str
    last_name: str

class EmployeeOut(Schema):
    id: int
    first_name: str
    last_name: str
```

```
# Model schemas
class EmployeeIn(ModelSchema):
    class Meta:
        model = Employee
        fields = ['first_name', 'last_name']

class EmployeeOut(Schema):
    class Meta:
        model = Employee
        fields = ['id', 'first_name', 'last_name']
```

ℹ️ Prashanth Rao : Pydantic benchmarks
Validation de 130K lignes en 0,155 secondes

Restreint Néosoft

# Django Ninja – Raisons d'être 3/

Les points de terminaison (*endpoints*) sont faciles à implémenter et l'asynchrone est possible :

```python
@api.post("/employee")
def create_employee(request, payload: EmployeeIn, response=EmployeeOut):
    payload_dict = payload.dict()
    employee = Employee.objects.create(**payload_dict)
    return employee

@api.get("/employee/{employee_id}", response=EmployeeOut)
def get_employee(request, employee_id: int):
    employee = get_object_or_404(Employee, id=employee_id)
    return employee

@api.get("/employee", response=list[EmployeeOut])
def list_employees(request):
    qs = Employee.objects.all()
    return qs
```

```python
@api.post("/employees")
async def create_employee(request, payload: EmployeeIn):
    payload_dict = payload.dict()
    employee = await Employee.objects.acreate(**payload_dict)
    return {"id": employee.id}

@api.get("/employees/{employee_id}", response=EmployeeOut)
async def get_employee(request, employee_id: int):
    employee = await aget_object_or_404(Employee, id=employee_id)
    return employee

@api.get("/employees", response=list[EmployeeOut])
async def list_employees(request):
    qs = Employee.objects.all()
    return [employee async for employee in qs]
```

Restreint Néosoft

# Mise en pratique

# 4

# Mise en pratique

néosoft

**Objectifs** :

1. Gérer une architecture de code simple et efficace (domaines et MVC)
2. Modéliser et gérer les migrations
3. Avoir un admin
4. Avoir une API Ninja et gérer ses routes
5. Mettre en place CRUD sur un modèle (Ninja)
6. Gérer une authentification simple
7. Gérer une pagination simple
8. Exécuter un serveur web : manage.py & gunicorn

Restreint Néosoft

# Mise en pratique : création du projet

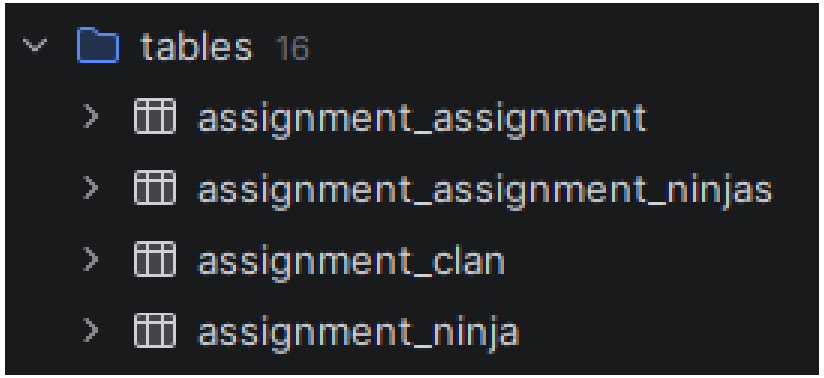
```
uv init
uv add django django-ninja
uv run django-admin startproject ninja_demo

cd ninja_demo

uv run ./manage.py migrate
uv run ./manage.py createsuperuser --username ninja --email ninja@python.org


uv run ./manage.py startapp assignment
```

# Mise en pratique : création des modèles Django

```python
# ninja_demo/assignment/models.py

from django.db import models


class Ninja(models.Model):
    """A spy for hire"""

    codename = models.CharField(max_length=30, unique=True)
    level = models.PositiveIntegerField(default=0)
    daily_rate = models.PositiveIntegerField(default=0)
    skills = models.CharField(max_length=200, blank=True)



class Clan(models.Model):
    """A japanese clan with some evil plans - or not"""

    name = models.CharField(max_length=20, unique=True)



class Assignment(models.Model):
    """A task assigned by a clan to a ninja or group of ninjas"""


    clan = models.ForeignKey(Clan, on_delete=models.CASCADE, related_name='assignments')
    ninjas = models.ManyToManyField(Ninja, related_name='assignments')
    goals = models.CharField(max_length=200, blank=True)
```

Ajouter 'assignment' aux INSTALLED_APPS

```
uv run ./manage.py migrate
```

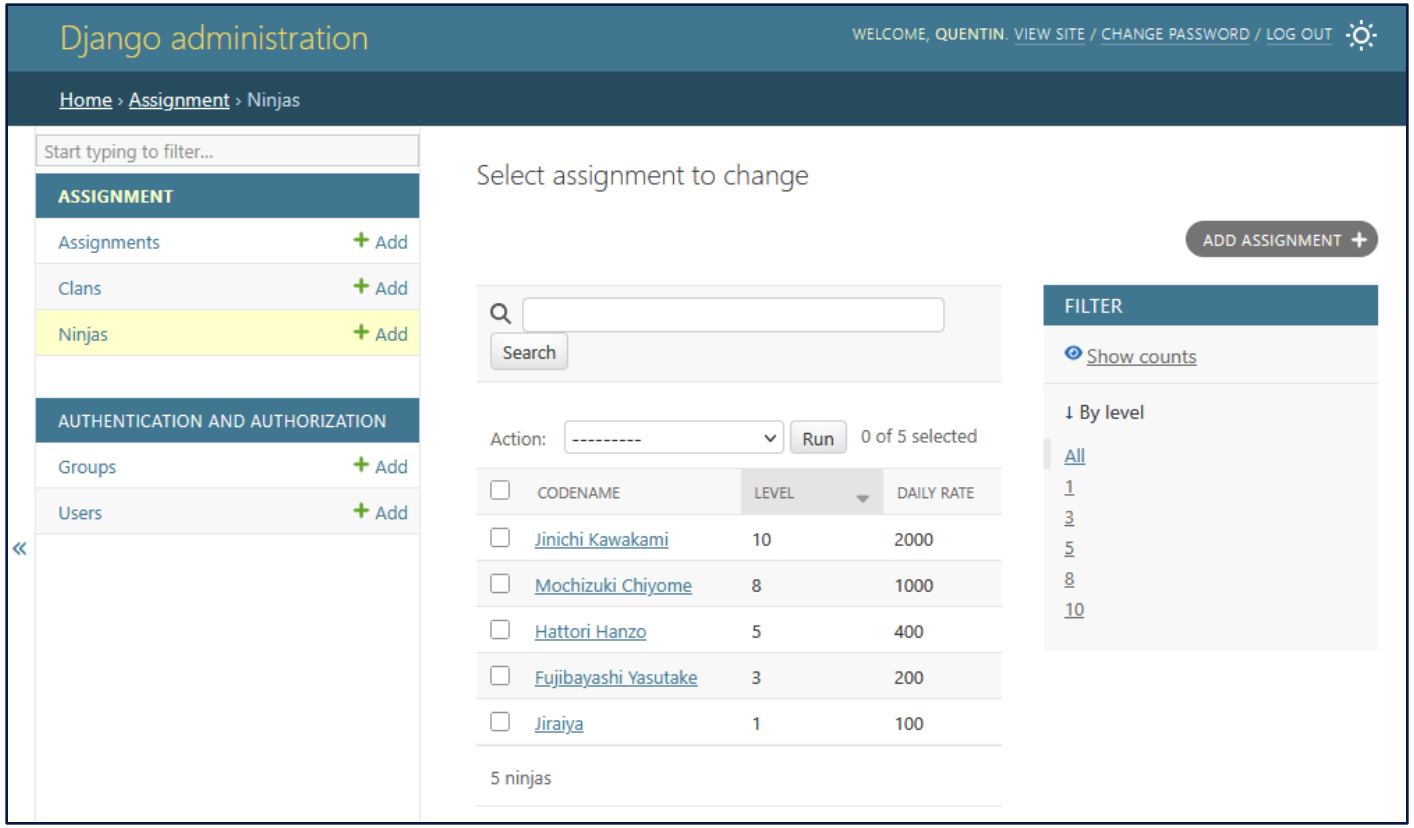



Restreint Néosoft

# Mise en pratique : administration

```python
# ninja_demo/assignment/admin.py

from django.contrib import admin

from .models import Ninja, Clan, Assignment


@admin.register(Ninja)
class NinjaAdmin(admin.ModelAdmin):
    list_display = ('codename', 'level', 'daily_rate',)
    list_filter = ('level',)
    search_fields = ('codename',)

@admin.register(Clan)
class ClanAdmin(admin.ModelAdmin):
    list_display = ('name', )
    list_filter = ('name',)
    search_fields = ('name',)

@admin.register(Assignment)
class AssignmentAdmin(admin.ModelAdmin):
    list_display = ('clan', 'pretty_ninjas', 'goals')
    list_filter = ('clan',)
    search_fields = ('clan__name',)
```

```
uv run ./manage.py runserver
```



# Mise en pratique : API et endpoints

```python
# ninja_demo/assignment/api/endpoints.py

from django.shortcuts import get_object_or_404
from ninja import Router

from .schemas import NinjaOut
from ..models import Ninja


# Create main router
router = Router()

# Create sub routers
ninja_router = Router(tags=['ninja'])

# Add sub routers to main router
router.add_router('ninja', ninja_router)

@ninja_router.post("/", response=NinjaOut)
def create_ninja(request, payload: NinjaIn):
    """Create a new ninja"""
    ninja = Ninja.objects.create(**payload.dict())
    return ninja
```

```python
# ninja_demo/ninja_demo/api.py

from ninja import NinjaAPI

from assignment.api.endpoints import router


api = NinjaAPI()
api.add_router("/assignment/", router)
```

```python
# ninja_demo/ninja_demo/urls.py

from django.contrib import admin
from django.urls import path


from .api import api


urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/', api.urls),
]
```

# Mise en pratique : schémas

```python
# ninja_demo/assignment/api/schemas.py

from ninja import ModelSchema

from ..models import Ninja


class NinjaIn(ModelSchema):
    class Meta:
        model = Ninja
        fields = ('codename', 'level', 'skills', 'daily_rate')


class NinjaOut(ModelSchema):
    class Meta:
        model = Ninja
        fields = ('id', 'codename', 'level', 'skills', 'daily_rate')
```

Restreint Néosoft

# Doc OpenAPI automatique

NinjaAPI 1.0.0 OAS 3.1

/api/openapi.json

## ninja

| | | |
|---|---|---|
| POST | /api/assignment/ninja/ | Create Ninja |
| POST | /api/assignment/ninja/list/ | List Ninjas |
| GET | /api/assignment/ninja/{ninja_id}/ | Get Ninja |
| PATCH | /api/assignment/ninja/{ninja_id}/ | Update Ninja |
| DELETE | /api/assignment/ninja/{ninja_id}/ | Delete Ninja |

## Schemas

NinjaOut > Expand all object

NinjaIn > Expand all object

NinjaFilter > Expand all object

NinjaInPatch > Expand all object

Votre spec est prête :
/api/openapi.json

# Mise en pratique : authentification simple

```python
# ninja_demo/ninja_demo/auth.py

from ninja.security import HttpBasicAuth

from django.contrib.auth import login, authenticate


class BasicAuth(HttpBasicAuth):
    def authenticate(self, request, username, password) -> str | None:
        user = authenticate(username=username, password=password)

        if user is not None:
            login(request, user)
            return username

        return None




api = NinjaAPI(auth=BasicAuth())
```

# Mise en pratique : pagination

```python
from ninja.pagination import paginate, PaginationBase
from ninja import Schema


class CustomPagination(PaginationBase):
    # only `skip` param, defaults to 5 per page
    class Input(Schema):
        skip: int

    class Output(Schema):
        items: List[Any] # `items` is a default attribute
        total: int
        per_page: int

    def paginate_queryset(self, queryset, pagination: Input, **params):
        skip = pagination.skip
        return {
            'items': queryset[skip : skip + 5],
            'total': queryset.count(),
            'per_page': 5,
        }


@api.get('/users', response=List[UserSchema])
@paginate(CustomPagination)
def list_users(request):
    return User.objects.all()
```

# Conclusions

# 5

# Avantages, points positifs

- Montée en compétence rapide, développement aisé
- Nombreuses fonctionnalités (filtres, pagination, authentification)
- Génération automatique de doc, facile à personnaliser
- Application de règles au niveau API, routeur, endpoint
- Bonne documentation - MàJ souhaitable
- Une bonne alternative à FastAPI - évolution ?

# Inconvénients, points négatifs, irritants

néosoft

- Nombreuses fonctionnalités

- Class-Based-Views (CBV) nativement
  - ❌ Django Ninja ➡️ Django Ninja Extra
  - ✔️ Django et DRF

- Gestion des permissions sur les endpoints, routeurs et API
  - ❌ Django Ninja ➡️ Django Ninja Extra
  - ✔️ Django et DRF

- Pas d'injection de dépendances : c'est voulu, en réf. à FastAPI
  - ❌ Django Ninja ➡️ Django Ninja Extra avec injector

- Authentification
  - Django Ninja offre des choses, pour une SPA : django-allauth

Restreint Néosoft

Fin

Questions